

METALNET: DEPENDENCY-FREE C++ CNN HEADER-ONLY LIBRARY

A PROJECT REPORT SUBMITTED
IN PARTIAL FULFILMENT OF THE REQUIREMENT
FOR THE AWARD OF THE DEGREE
OF

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE & ENGINEERING



BY

Raj Kunwar Prabhat 22050440078

Mayan Kumar 22050440058

Sujeet Kumar 22050440097

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CAMBRIDGE INSTITUTE OF TECHNOLOGY**

TATISILWAI, RANCHI – 835103

JHARKHAND, INDIA

2026

DECLARATION CERTIFICATE

This is to certify that the work presented in the project entitled “**MetalNet: Dependency-Free C++ CNN Header-Only Library**” in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering at Cambridge Institute of Technology, Tatisilwai, Ranchi is an authentic work carried out under my supervision and guidance.

To the best of my knowledge, the content of this project does not form a basis for the award of any previous Degree to anyone else.

Date: _____

PROF. DEEPAK KUMAR VERMA

Dept. of Computer Science and Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

PROF. DEEPAK KUMAR VERMA
(Head of Department)

Dept. of Computer Science and Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

CERTIFICATE OF APPROVAL

The foregoing project entitled “MetalNet: Dependency-Free C++ CNN Header-Only Library” is hereby approved as a creditable work on the topic and has been presented satisfactorily to warrant its acceptance as a prerequisite to the degree for which it has been submitted.

It is understood that by this approval, the undersigned does not necessarily endorse any conclusion drawn or opinion expressed therein, but approves the project for the purpose for which it is submitted.

(Internal Examiner)

(External Examiner)

HEAD OF DEPARTMENT

Dept. of Computer Science and Engineering

Cambridge Institute of Technology

Tatisilwai, Ranchi – 835103

ACKNOWLEDGEMENT

I take this opportunity to thank all who have contributed towards shaping this Final Year Project Report. I would like to express my sincere thanks to Prof. Deepak Kumar Verma (HOD, Department of Computer Science & Engineering) and to my guide Prof. Deepak Kumar Verma, whose help in the course of development of this manuscript is invaluable. As my supervisor, he/she has constantly encouraged me to remain focused on achieving my goal.

I would also like to thank the technical staff members of the Department who have been kind enough to advise and help in their respective roles. I have been fortunate to have a wonderful support structure among fellow students. My sincere thanks to everyone who has provided me with kind words, new ideas, useful criticism, or their valuable time. I am truly indebted. Last but not least, I would like to dedicate this work to my family for their love and understanding.

Mayan Kumar
22050440058

ABSTRACT

Deep learning vision systems typically rely on large frameworks like PyTorch or TensorFlow, which carry significant dependency overhead and are designed primarily for GPUs. On CPU-only environments such as laptops or edge devices, these frameworks introduce unnecessary resource consumption. This project presents **MetalNet**, a high-performance Convolutional Neural Network (CNN) library written from scratch in pure C++20. MetalNet is fully **header-only** and **dependency-free**, requiring no installation, linking, or configuration, and is designed specifically to optimize CPU execution.

Despite its lightweight design, MetalNet achieves high performance by treating the CPU as a first-class citizen. It implements an **Implicit GEMM** convolution algorithm that avoids memory-heavy *im2col* buffering, and stores data in the cache-friendly **NHWC** layout. Innermost arithmetic operations are accelerated using hand-written **AVX2 and FMA** SIMD instructions, while loop tiling and parallelization via **OpenMP** optimize multi-core CPU scaling. Together, these optimizations yield a **57× speedup** in convolution operations over a naive scalar baseline.

MetalNet supports both training and inference. It features a Directed Acyclic Graph (DAG) based automatic differentiation engine, core layers (Convolutional, Batch Normalization, Pooling, Dense), activation functions, SGD and Adam optimizers, and a built-in profiler. Tested against PyTorch on LeNet-5, VGG-style, and ResNet-18 architectures, MetalNet consistently runs **1.6–2.4× faster** for inference on the same CPU while reducing peak memory consumption by approximately **70%**. MetalNet demonstrates that high-performance deep learning is achievable on standard CPUs without complex external dependencies.

Keywords: Convolutional Neural Networks, C++20, Header-Only Library, SIMD Optimization, Implicit GEMM, Automatic Differentiation, AVX2, Cache Optimization, OpenMP, Deep Learning, CPU Performance

TABLE OF CONTENTS

Declaration Certificate	ii
Certificate of Approval	iii
Acknowledgement	iv
Abstract	v
Table of Contents	vi
List of Figures	x
List of Tables	x
List of Abbreviations	xi
CHAPTER 1 – INTRODUCTION	1
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Objectives of the Project	3
1.4 Scope of the Project	3
1.4.1 Inclusions	3
1.4.2 Exclusions	4
1.4.3 Target Users and Environment	4
1.5 Organization of the Report	5
CHAPTER 2 – LITERATURE REVIEW	6
2.1 Related Works	6
2.1.1 Deep Learning Frameworks	6
2.1.2 Convolution Optimization Algorithms	6
2.1.3 SIMD and Cache Optimization	7
2.1.4 Automatic Differentiation	7
2.1.5 Batch Normalization	7
2.1.6 Optimization Algorithms	7
2.2 Comparative Analysis	8
2.3 Gap Identification and Project Contribution	8
2.3.1 Why CPU Optimization Matters	9
CHAPTER 3 – SYSTEM ANALYSIS AND DESIGN	10
3.1 System Requirements	10

3.1.1	Hardware Requirements	10
3.1.2	Software Requirements	10
3.2	System Architecture	10
3.2.1	Core Tensor Engine	11
3.2.2	Layer System	11
3.2.3	Optimization System	11
3.2.4	Profiler and Utilities	12
3.3	Data Flow Diagrams	12
3.3.1	Level 0 - Context Diagram	12
3.3.2	Level 1 - Detailed Data Flow	13
3.4	Design Patterns and Implementation Details	13
3.4.1	Header-Only Design	13
3.4.2	RAII (Resource Acquisition Is Initialization)	13
3.4.3	Template Metaprogramming	13
3.5	Memory Layout and Optimization	14
CHAPTER 4 – IMPLEMENTATION		15
4.1	Technologies and Tools Used	15
4.1.1	Programming Language: C++20	15
4.1.2	Build System: CMake	15
4.1.3	SIMD Libraries	16
4.1.4	Parallelization: OpenMP	16
4.1.5	Profiling and Visualization	16
4.2	Module Description	17
4.2.1	Tensor Module	17
4.2.2	Layer Module	17
4.2.3	Conv2D Module: Implicit GEMM Implementation	18
4.2.4	Automatic Differentiation Engine	19
4.2.5	Optimization Module	20
4.3	Algorithm / Methodology	21
4.3.1	Implicit GEMM Convolution Algorithm	21
4.3.2	Cache-Aware Loop Tiling	22
4.3.3	SIMD Vectorization	23
4.3.4	Adaptive Multithreading	23
4.4	Code Implementation Highlights	24
4.4.1	FusedConvBNReLU Layer	24
4.4.2	Dropout Layer	25
4.4.3	Memory Optimization Techniques	25
4.5	Integration with Standard Frameworks	26

CHAPTER 5 – TESTING AND RESULTS	28
5.1 Testing Strategy	28
5.1.1 Unit Testing	28
5.1.2 Numerical Gradient Checking	28
5.1.3 Integration Testing	28
5.1.4 Performance Testing	28
5.2 Test Cases and Results	29
5.2.1 Unit Test Results	29
5.2.2 Integration Test Results	29
5.3 Performance Analysis	30
5.3.1 Inference Latency	30
5.3.2 Memory Usage	31
5.3.3 Scaling Analysis	31
5.3.4 Multi-threaded Scaling	32
5.3.5 Component-Level Performance Analysis	33
5.3.6 Comparison with Other Frameworks	33
5.3.7 Scaling Behavior with Model Complexity	34
CHAPTER 6 – CONCLUSION AND FUTURE WORK	35
6.1 Conclusion	35
6.2 Limitations and Current Constraints	36
6.3 Future Enhancements	36
6.3.1 Short-Term (6-12 months)	36
6.3.2 Medium-Term (1-2 years)	37
6.3.3 Long-Term (2+ years)	37
6.4 Lessons Learned	37
6.5 Design Trade-offs and Justification	38
6.5.1 Header-Only vs. Separate Compilation	38
6.5.2 NHWC Layout vs. NCHW	38
6.5.3 CPU-Only vs. GPU Support	38
6.6 Reproducibility and Open Science	39
6.7 Broader Impact and Applications	39
6.7.1 Edge Intelligence	39
6.7.2 Real-Time Systems	39
6.7.3 Privacy-Preserving Inference	40
6.8 Final Remarks	40
6.9 Contributions Summary	40
References	41
Appendix A – Source Code	43

A.1	Core Tensor Class	43
A.2	Conv2D Layer Implementation	45
A.3	SGD Optimizer Implementation	46
Appendix B – Screenshots / Sample Outputs		47
B.1	Output Screenshots	48

LIST OF FIGURES

Figure 3.1	MetalNet system architecture	12
Figure 3.2	Data Flow Diagram: Level 0 (Context Diagram)	12
Figure 3.3	Data Flow Diagram: Level 1 (Training Pipeline)	13
Figure 5.1	Inference latency comparison	30
Figure 5.2	Peak inference memory comparison	31
Figure 5.3	VGG-16 inference latency versus batch size	32
Figure 5.4	OpenMP scaling efficiency	32
Figure 5.5	Cumulative speedup of layered optimizations	33
Figure B.1	Filter Outputs	48
Figure B.2	Inference Output	48
Figure B.3	Profiler Interface	49
Figure B.4	Training Progress	49

LIST OF TABLES

Table 2.1	Comparison of CNN Frameworks and Optimization Approaches	8
Table 3.1	Hardware Requirements for MetalNet	10
Table 3.2	Software Requirements for MetalNet	10
Table 5.1	Unit Test Results for Core Layers	29
Table 5.2	MNIST Training Results (10 epochs)	29
Table 5.3	CIFAR-10 Training Results (20 epochs)	30
Table 5.4	Inference Latency Comparison (batch size 32, ms per batch)	30
Table 5.5	Memory Footprint Comparison (MB, batch size 32)	31
Table 5.6	Multi-threading Scaling (VGG-16, batch size 32)	32
Table 5.7	Performance Contribution of Individual Optimizations	33
Table 5.8	Comprehensive Framework Comparison	34
Table 5.9	Performance vs Model Complexity	34

LIST OF ABBREVIATIONS

CNN	Convolutional Neural Network
GEMM	General Matrix Multiplication
SIMD	Single Instruction, Multiple Data
AVX2	Advanced Vector Extensions 2
FMA	Fused Multiply–Add
im2col	Image to Column
DAG	Directed Acyclic Graph
SGD	Stochastic Gradient Descent
BN	Batch Normalization
ReLU	Rectified Linear Unit
NHWC	Batch, Height, Width, Channels
NCHW	Batch, Channels, Height, Width
L1/L2	Level 1 / Level 2 Cache
OpenMP	Open Multi-Processing
GPU	Graphics Processing Unit
CPU	Central Processing Unit
MNIST	Modified NIST handwritten digit dataset
CIFAR	Canadian Institute For Advanced Research
VGG	Visual Geometry Group
API	Application Programming Interface
DFD	Data Flow Diagram
UML	Unified Modelling Language
UAT	User Acceptance Testing
IDE	Integrated Development Environment